

The Advanced Product Manager Course

From 0-to-1 Scoping to Enterprise Portfolio Management

A Comprehensive Deep-Dive Manual

Edition 2026

Contents

1	0-to-1 Product Development & Project Scoping	2
1.1	The Genesis: Ideation and Feasibility Studies	2
1.2	Mastering the PRD (Product Requirements Document)	2
1.2.1	Anatomy of an Advanced PRD	2
1.3	Scope Management & Scope Creep	3
2	Financial Engineering, Capital Budgeting & ROI	5
2.1	Capital Budgeting: NPV and IRR	5
2.1.1	Net Present Value (NPV)	5
2.1.2	Internal Rate of Return (IRR)	5
2.2	Project Finance & Cost Management	6
3	The Engine: Execution, Jira Masterclass & Timelines	8
3.1	The Jira Masterclass: 0 to 1 for PMs	8
3.1.1	The Agile Hierarchy	8
3.1.2	JQL (Jira Query Language) for PMs	8
3.2	Timelines, PERT & CPM (Critical Path Method)	9
3.2.1	PERT (Program Evaluation and Review Technique)	9
3.2.2	Critical Path Method (CPM)	9
4	Risk Management, GRC & Engineering Tech Debt	10
4.1	Risk Management Framework (ISO 31000)	10
4.1.1	Risk Analysis Matrix	10
4.2	GRC (Governance, Risk Management, and Compliance)	10
4.3	Engineering Management: The Tech Debt Crisis	10
5	Stakeholder Management & Procurement	12
5.1	Stakeholder Analysis (The Power/Interest Grid)	12
5.2	Contract Management & Procurement	12
6	Program & Portfolio Management	14
6.1	Project vs. Program vs. Portfolio	14
6.2	Portfolio Resource Allocation	14
6.3	Final Exam / Capstone Challenge	15

Chapter 1

0-to-1 Product Development & Project Scoping

1.1 The Genesis: Ideation and Feasibility Studies

Building a product from “0 to 1” (creating something entirely new) is fundamentally different from “1 to n” (scaling an existing product). In the 0-to-1 phase, you are searching for product-market fit.

Before a single line of code is written, a PM must conduct rigorous **Feasibility Studies**. This is not a box-checking exercise; it is the shield that prevents your company from burning millions of dollars.

- **Technical Feasibility:** Can our current engineering stack support this? Do we need machine learning capabilities we don't possess? *Real-world consequence:* Promising a real-time collaborative editor (like Google Docs) without realizing your backend relies on slow, batch-processing relational databases.
- **Financial Feasibility:** What is the Customer Acquisition Cost (CAC) vs. Lifetime Value (LTV)? If $CAC > LTV$, the product is mathematically doomed.
- **Market Feasibility:** Is the Total Addressable Market (TAM) large enough to justify the R&D?

1.2 Mastering the PRD (Product Requirements Document)

The PRD is the PM's bible. A weak PRD leads to endless meetings, frustrated engineers, and scope creep. An advanced PRD leaves no room for ambiguity.

1.2.1 Anatomy of an Advanced PRD

1. **The North Star Metric:** What is the single metric that defines success? (e.g., Spotify: “Time spent listening”).
2. **Counter-Metrics:** What metric must we ensure we *do not* hurt? (e.g., We want to increase ad revenue, but our counter-metric is “User churn rate”).
3. **User Stories & Acceptance Criteria:** Written in BDD (Behavior-Driven Development) format: *Given [context], When [action], Then [outcome]*.

4. **Out of Scope (Crucial):** Explicitly stating what you will **not** build. This prevents engineers from over-engineering solutions.

Visual Framework: The PRD Structure Template

Section	Content / Purpose
1. Executive Summary	2-sentence pitch. Why are we doing this <i>now</i> ?
2. Success Metrics	Primary metric, secondary metrics, and counter-metrics.
3. Target Audience	Specific user personas and their pain points.
4. Functional Reqs	The actual features, prioritized (P0, P1, P2).
5. Non-Functional Reqs	Latency (<200ms), uptime (99.99%), security standards.
6. Out of Scope	Features intentionally deferred to V2.

Concept Check & Strategic Application

Situation 1: You are writing a PRD for a new checkout flow. The engineers ask if they should build a custom payment gateway or integrate Stripe. You realize this isn't in the PRD. **Question:** Which section of the PRD failed to address this, and what is the strategic implication?

- A. The Success Metrics section; it affects the conversion rate.
- B. The Functional Requirements; it dictates the buy-vs-build strategy.
- C. The Target Audience; different users prefer different gateways.
- D. The Out of Scope section; custom gateways should be explicitly deferred.

Detailed Answer Key & Explanation

Answer: B & D. While functionally it is a buy-vs-build decision (B), advanced PMs know that building a payment gateway from scratch is a massive undertaking. It should have been explicitly listed in the **Out of Scope (D)** section as "Custom Payment Gateway (Will use Stripe API for MVP)" to prevent engineering from even considering building it. *Consequence of missing this:* Engineers spend 3 sprints building PCI-compliant infrastructure instead of launching the product.

1.3 Scope Management & Scope Creep

Scope Management is the process of defining what is included in the project and, more importantly, controlling changes to that baseline.

How to handle Scope Creep in reality: When the CEO walks over and says, "Can we just add this one AI feature before launch?", an advanced PM does not say "No." An advanced PM says: "Yes, we can do that. It will cost us exactly 3 weeks of delay on the core release, and we will have to drop the reporting dashboard. Should I update the roadmap?" *Make the trade-off visible.*

Real-World Case Study

Case Study 1: The Bloated MVP

Context: You are the Lead PM for “HealthSync,” a 0-to-1 app designed to aggregate data from Apple Watch, Fitbit, and Garmin into one dashboard for doctors. The initial scope (P0) was just Apple Watch integration.

The Crisis: Midway through Sprint 3, the VP of Sales promises a massive hospital client that Fitbit integration will be included in the V1 launch, which is exactly 4 weeks away. Engineering states Fitbit integration will take 6 weeks of dedicated work. The CEO tells you to “make it happen.”

Question 1 (MCQ): What is your immediate first step as the PM?

- A. Tell engineering to work weekends to meet the CEO’s demand.
- B. Update the PRD to include Fitbit and extend the launch date by 2 weeks.
- C. Conduct an impact analysis on cost, timeline, and risk, and present options to the CEO.
- D. Tell the VP of Sales to retract the promise to the client.

Question 2 (Strategic): If you are forced to include Fitbit in 4 weeks, what PM techniques can you use to manipulate the “Iron Triangle” (Scope, Time, Cost) to achieve this?

Answers & Analysis: Q1 Answer: C. Never blindly accept scope changes (A, B) or immediately cause a political war (D). You must quantify the impact. If it takes 6 weeks, you need to show the CEO what will break if you try to compress it to 4.

Q2 Strategic Answer: To fit a 6-week feature into 4 weeks, you must alter the other variables:

- **Reduce Scope (Micro-scoping):** Don’t pull *all* Fitbit data. Only pull “Heart Rate” for V1. De-scope Sleep and Steps.
- **Increase Cost (Resources):** Pull two backend engineers from another non-critical project (like the internal reporting tool) to parallelize the API integration.
- **Accept Technical Debt (Risk):** Hardcode certain data mappings temporarily instead of building a scalable parsing engine, with a strict Jira ticket to refactor it in V1.1.

Chapter 2

Financial Engineering, Capital Budgeting & ROI

A PM who cannot speak the language of finance will never become a Product Executive. You must justify your product's existence through financial metrics.

2.1 Capital Budgeting: NPV and IRR

When proposing a new product, the CFO will ask: "Is this worth our capital?" You answer using Capital Budgeting techniques.

2.1.1 Net Present Value (NPV)

Money today is worth more than money tomorrow (Time Value of Money). NPV calculates the current value of all future cash flows of your product, minus the initial investment.

$$NPV = \sum_{t=1}^n \frac{C_t}{(1+r)^t} - C_0$$

Where: C_t = Net cash inflow during the period t .

r = Discount rate (the company's required rate of return).

t = Number of time periods.

C_0 = Total initial investment costs.

Rule of Thumb: If $NPV > 0$, the project adds value to the company.

2.1.2 Internal Rate of Return (IRR)

IRR is the discount rate (r) that makes the NPV of all cash flows equal to zero. It represents the annualized effective compounded return rate of your product. If $IRR >$ the company's Cost of Capital, you fund the product.

Concept Check & Strategic Application

Financial Calculation Problem You are pitching a new Enterprise Tier for your SaaS product. - Upfront R&D Cost (Year 0): \$500,000 - Projected Net Cash Flow Year 1: \$200,000 - Projected Net Cash Flow Year 2: \$300,000 - Projected Net Cash Flow Year 3: \$400,000 - Company Discount Rate: 10%

Question: Is this product financially viable based on NPV?

Detailed Answer Key & Explanation

Let's calculate the Present Value (PV) of each year's cash flows: - Year 1 PV: $\$200,000 / (1 + 0.10)^1 = \$181,818$ - Year 2 PV: $\$300,000 / (1 + 0.10)^2 = \$247,933$ - Year 3 PV: $\$400,000 / (1 + 0.10)^3 = \$300,525$

Total PV of Inflows = $\$181,818 + \$247,933 + \$300,525 = \$730,276$

NPV = Total PV - Initial Cost = $\$730,276 - \$500,000 = \$230,276$. **Conclusion:** Because the NPV (\$230,276) is significantly greater than 0, the project is highly viable and creates value for the shareholders.

2.2 Project Finance & Cost Management

As a PM, you manage OPEX (Operational Expenditure) and CAPEX (Capital Expenditure).

- **CAPEX:** Buying servers, software licenses, patents.
- **OPEX:** Cloud hosting costs (AWS/GCP), API usage fees, marketing spend, support staff.

Advanced PM Insight: Cloud computing shifted a lot of software costs from CAPEX to OPEX. You must monitor your AWS/GCP bills. A poorly optimized database query written by your engineers can cost the company thousands of dollars in OPEX per month.

Real-World Case Study

Case Study 2: The CFO's Dilemma

Context: You manage a video streaming product. You have budget for only one major initiative this year. - **Initiative A (New Feature):** Implement "Watch Party" mode. Expected to generate \$1.2M in new subscription revenue over 2 years. Costs \$400k to build. - **Initiative B (Tech Debt):** Migrate the video transcoding pipeline to a more efficient modern architecture. Will generate \$0 in new revenue, but will save \$50k per month in AWS cloud costs indefinitely. Costs \$300k to build.

Question 1 (MCQ): From a purely financial ROI perspective over a 2-year horizon, which project is better?

- A. Initiative A
- B. Initiative B
- C. They are exactly equal.
- D. Cannot be determined.

Question 2 (Strategic): How do you pitch Initiative B (Tech Debt) to a growth-obsessed CEO?

Answers & Analysis: Q1 Answer: B. Let's do the math (ignoring discount rates for simplicity). - Initiative A ROI: $(\$1.2\text{M} - \$400\text{k}) = +\$800\text{k}$ over 2 years. - Initiative B ROI: Savings = $\$50\text{k} * 24 \text{ months} = \1.2M . Net benefit = $(\$1.2\text{M} - \$300\text{k}) = +\$900\text{k}$. Initiative B is actually more profitable over 2 years and has significantly less market risk (saving money is guaranteed, new subscriptions are speculative).

Q2 Strategic Answer: Never pitch tech debt as "refactoring code." Pitch it as **Margin Expansion**. Tell the CEO: "Initiative B increases our gross margins by 15%, immediately improving our EBITDA without needing to acquire a single new customer, giving us more runway to fund future growth features."

Chapter 3

The Engine: Execution, Jira Masterclass & Timelines

Ideas are cheap; execution is everything. This chapter covers the exact mechanics of how a PM drives engineering teams to deliver software.

3.1 The Jira Masterclass: 0 to 1 for PMs

Jira is the industry standard for issue tracking and Agile project management. Using Jira poorly turns it into a chaotic to-do list. Using it well turns it into an automated reporting engine.

3.1.1 The Agile Hierarchy

- **Epic:** A large body of work (e.g., "Implement User Authentication"). Takes multiple sprints.
- **Story:** A single feature delivering value (e.g., "As a user, I want to log in via Google").
- **Task:** Technical work not directly yielding user value (e.g., "Upgrade database to v14").
- **Bug:** A defect in production or staging.
- **Sub-task:** Breaking down a Story into dev/QA steps (e.g., "Design DB schema," "Write API endpoint").

Visual Framework: Advanced Jira Kanban Board Layout

A professional PM structures their board to map the exact SDLC (Software Development Life Cycle).

[Backlog] → [To Do (Sprint)] → [In Progress] → [Code Review] → [QA/Testing] → [Done]

WIP Limits (Work In Progress): An advanced technique is setting a limit (e.g., max 3 tickets) on the "Code Review" column. If the limit is hit, developers *must* stop writing new code and review their peers' code to unblock the pipeline.

3.1.2 JQL (Jira Query Language) for PMs

You need data fast. JQL is how you interrogate Jira.

- **Find neglected bugs:** `project = APP AND type = Bug AND status = Open AND createdAt < -30d`

- **Find scope creep:** `sprint in openSprints() AND createdAt > startOfDay()` (Finds tickets added to the sprint *after* it started).

3.2 Timelines, PERT & CPM (Critical Path Method)

3.2.1 PERT (Program Evaluation and Review Technique)

Engineers are notoriously bad at estimating time. They give you a single point estimate ("It'll take 3 days"). Advanced PMs force 3-point estimation to account for uncertainty:

- **O (Optimistic):** If everything goes perfectly.
- **P (Pessimistic):** If everything goes wrong.
- **M (Most Likely):** The realistic guess.

$$\text{Expected Time (TE)} = \frac{O + 4M + P}{6}$$

Example: A dev says a feature will take 5 days (M). But optimistically it could be 3 days (O), and if the API documentation is wrong, it could take 13 days (P). $TE = (3 + 20 + 13)/6 = 36/6 = \mathbf{6 \text{ days}}$. You plan for 6 days, not 5.

3.2.2 Critical Path Method (CPM)

The Critical Path is the longest sequence of dependent tasks in a project. If any task on the critical path is delayed, the **entire project is delayed**. Tasks not on the critical path have "Float" or "Slack" (they can be delayed without affecting the final launch date).

Real-World Case Study

Case Study 3: The 3-Month Delay

Context: You are building an e-commerce platform. The project has 3 concurrent streams: 1. Frontend UI Design (Takes 4 weeks) 2. Payment Gateway Integration (Takes 8 weeks) 3. Database Setup & Inventory Logic (Takes 10 weeks) After these are done, Final QA testing takes 2 weeks. Total planned time: 12 weeks (10 weeks max path + 2 weeks QA).

The Crisis: At week 2, the UI Designer quits. It will take 4 weeks to hire a new one, and then 4 weeks to do the work. The Marketing team panics, saying "The project is delayed by 4 weeks!"

Question (Strategic): Are they right? What do you tell the CEO using CPM principles?

Answer & Analysis: Marketing is wrong. You tell the CEO: "The project launch date is **not** delayed."

Why? You look at the Critical Path. The longest path is the Database Setup (10 weeks). The UI Design was originally only 4 weeks, meaning it had **6 weeks of float/slack** ($10 - 4 = 6$). Even if the UI design is delayed by 4 weeks (hiring) + 4 weeks (work) = 8 weeks total, it is still shorter than the 10-week Database path. The UI team will still finish 2 weeks before the Database team.

PM Action: You confidently manage the stakeholder panic by proving the delay is absorbed by the project's slack.

Chapter 4

Risk Management, GRC & Engineering Tech Debt

4.1 Risk Management Framework (ISO 31000)

Risk is the effect of uncertainty on objectives. Advanced PMs don't just react to problems; they preempt them using a **Risk Register**.

4.1.1 Risk Analysis Matrix

You assess risks based on two dimensions: **Probability** (Likelihood of happening) and **Impact** (Damage if it happens).

- **High Prob / High Impact:** Mitigate immediately (e.g., Servers crashing on Black Friday).
- **Low Prob / High Impact:** Transfer or insure against (e.g., Data center catches fire).
- **High Prob / Low Impact:** Accept or mitigate cheaply (e.g., Minor UI bugs).

Risk Responses: 1. **Avoid:** Change the plan to eliminate the risk entirely. 2. **Mitigate:** Reduce the probability or impact. 3. **Transfer:** Shift the risk to a third party (e.g., buying cyber insurance, outsourcing payment processing to Stripe). 4. **Accept:** Acknowledge the risk and create a contingency fund.

4.2 GRC (Governance, Risk Management, and Compliance)

You cannot build software in a vacuum. You are bound by laws.

- **GDPR / CCPA:** Data privacy. Users have the "Right to be forgotten." If your database schema doesn't allow for hard-deleting user data cleanly, you are facing massive fines.
- **SOC 2:** Security and trust. If you sell B2B (to other businesses), they will demand a SOC 2 audit to prove your engineering practices are secure.

4.3 Engineering Management: The Tech Debt Crisis

Technical Debt is taking a shortcut now to ship faster, which results in extra work later. *The Consequence:* If a PM forces engineers to constantly ship features without allocating 20% of sprint

capacity to refactoring (paying down tech debt), the codebase becomes “spaghetti.” Within 18 months, feature velocity will drop to near zero. Every new feature will break three existing ones.

Concept Check & Strategic Application

Situation: You are building a healthcare app storing patient records. An engineer suggests using an open-source, unvetted database plugin to save 3 weeks of development time. **Question:** Which GRC principle is most at risk, and what is the PM’s correct response?

- A. Governance; approve it because speed to market is the primary governance metric.
- B. Compliance; reject it due to HIPAA/GDPR regulatory risks of unvetted data storage.
- C. Risk; accept the risk because 3 weeks of saved time equals lower OPEX.
- D. Management; approve it but document it in the Risk Register.

Detailed Answer Key & Explanation

Answer: B. In healthcare/fintech, Compliance (HIPAA in the US, GDPR in Europe) overrides speed to market. Using unvetted plugins for sensitive PII (Personally Identifiable Information) violates data security compliance. A data breach will bankrupt the company. You must reject the shortcut.

Real-World Case Study

Case Study 4: The Zero-Day Exploit

Context: You manage a B2B SaaS platform. On a Friday at 4 PM, a severe Zero-Day vulnerability is discovered in a core open-source library your software uses. Hackers are actively exploiting this globally.

The Crisis: Fixing this requires pulling all engineers off the current sprint, missing a major feature deadline promised to your biggest client.

Question (Strategic): Outline the step-by-step crisis management protocol balancing Engineering, Stakeholders, and GRC.

Answer & Analysis: 1. **Containment & Triage (Engineering):** Immediately halt all feature development. Direct the DevOps/SecOps team to apply the patch or sever the vulnerable microservice from the main database to prevent data exfiltration. 2. **Governance & Compliance (GRC):** Notify the Legal and Security compliance teams. If user data was compromised, GDPR requires notifying regulators within 72 hours. 3. **Stakeholder Management (Communication):** Draft a transparent incident report. Call the massive client directly. Do not hide the delay. Say: “We have halted feature delivery to protect your proprietary data from a global zero-day threat. Security is our P0. The feature is delayed by 1 week.” Clients forgive delays; they do not forgive data breaches.

Chapter 5

Stakeholder Management & Procurement

5.1 Stakeholder Analysis (The Power/Interest Grid)

A PM spends 50% of their time aligning stakeholders. Not all stakeholders are equal. You must map them.

Visual Framework: The Power / Interest Matrix

- **High Power, High Interest (Manage Closely):** The CEO, major investors, the VP of Engineering. You must consult them daily/weekly. Over-communicate.
- **High Power, Low Interest (Keep Satisfied):** The CFO, Legal. They don't care about daily features, but they can kill your project if you violate budget or laws. Give them high-level summaries.
- **Low Power, High Interest (Keep Informed):** Customer Support, Sales Reps. They need to know what's coming to do their jobs, but they don't get to dictate the roadmap. Send them release notes.
- **Low Power, Low Interest (Monitor):** General employees.

5.2 Contract Management & Procurement

When you decide to "Buy" instead of "Build" (e.g., buying a CRM, integrating a 3rd party AI API), you enter Procurement.

- **RFP (Request for Proposal):** A document you issue to vendors asking them to pitch how they can solve your problem and at what cost.
- **SLA (Service Level Agreement):** The legal contract defining performance. *Crucial PM Knowledge:* If an API vendor promises 99.9% uptime, that still allows for 43 minutes of downtime a month. If your app is mission-critical (like trading software), 43 minutes of downtime is catastrophic. You must negotiate a 99.99% or 99.999% SLA.

Real-World Case Study

Case Study 5: The Hostile VP of Sales

Context: You are the PM of a mature B2B product. You have a strict roadmap focused on reducing technical debt and improving core UX over the next 6 months.

The Crisis: The VP of Sales (High Power, High Interest) crashes your sprint planning. He says, "If we don't build a custom Blockchain ledger feature by next month, we lose a \$2 Million contract with Bank X. Drop everything and build it."

Question (Strategic): How do you handle this without destroying your roadmap or making an enemy of the VP of Sales?

Answer & Analysis: Never say "No" outright to a \$2M revenue driver, but never say "Yes" blindly. Use the **Trade-off Framework**. 1. **Validate the Request:** Speak to Bank X directly. Often, Sales misinterprets the requirement. Bank X might just want an immutable audit log, not an actual decentralized blockchain. An audit log takes 1 week; a blockchain takes 6 months. 2. **Present the Cost to the Executive Team:** If it is truly a massive feature, you take it to the Portfolio Board (CEO, CFO). You present: "To build Feature X for \$2M in revenue, we must delay UX improvements which will cause a projected 5% churn in our existing user base, costing us \$3M in ARR (Annual Recurring Revenue)." 3. **Force the Decision Upward:** You let the executives make the business decision based on your data. If they say do it, you document the pivot and adjust the roadmap.

Chapter 6

Program & Portfolio Management

As you move from a standard PM to a Group PM (GPM) or VP of Product, you no longer manage just one product. You manage an interconnected ecosystem.

6.1 Project vs. Program vs. Portfolio

- **Project Management:** Delivering a specific feature or app (e.g., Building the Uber Eats Driver App).
- **Program Management:** Managing a group of related projects coordinated to achieve benefits not available from managing them individually (e.g., Coordinating the Driver App, User App, and Restaurant Tablet to launch in a new city simultaneously).
- **Portfolio Management:** Managing a group of programs, projects, and operations to achieve strategic business objectives (e.g., Uber Rideshare vs. Uber Eats vs. Uber Freight). This is about **Resource Allocation**.

6.2 Portfolio Resource Allocation

At the portfolio level, you are an investor. You have a finite amount of engineering capital (e.g., 500 engineers). How do you deploy them?

- **Core (70% of resources):** Optimizing existing products that generate the current revenue.
- **Adjacent (20%):** Expanding existing business into new areas (e.g., taking a US product to Europe).
- **Transformational / Horizon 3 (10%):** High-risk, high-reward bets (0-to-1 innovations that could be the future of the company).

Concept Check & Strategic Application

Situation: You are the VP of Product. Your “Cash Cow” legacy product is losing 2% market share a year, but still funds the whole company. Your new AI-driven product is growing at 300% YoY but burning cash. The board wants you to cut engineering from the AI product to patch up the legacy product. **Question:** Using Portfolio theory, what is the strategic error in the board’s request?

Detailed Answer Key & Explanation

Answer: The board is exhibiting short-term risk aversion and violating the **Horizon Innovation Strategy**. Starving a high-growth “Horizon 3” product (the AI tool) to save a declining “Horizon 1” legacy product means the company is sacrificing its future for a temporary fix. As a Portfolio Manager, you must explain that the legacy product is in its “harvest” phase; it should operate on maintenance mode while capital is aggressively re-allocated to the AI product to capture the future market.

6.3 Final Exam / Capstone Challenge

Real-World Case Study

Capstone: The Enterprise PM Scenario

Context: You are the Group Product Manager for a FinTech company preparing for an IPO (Initial Public Offering). You manage a portfolio of 3 programs. You receive the following information on a Monday morning: 1. **Financial:** The CFO tells you EBITDA must increase by 10% this quarter for the IPO to succeed. 2. **Risk/GRC:** The Head of InfoSec informs you that a new European regulation requires a massive overhaul of how you store transaction data, due by the end of the quarter, or you face suspension of operations in the EU. 3. **Execution:** Your star engineering team on your highest ROI project just reported a 4-week delay due to underestimated technical debt.

Task: Construct your prioritization strategy. What do you do first, what gets delayed, and how do you manage the stakeholders?

Master-Level Analysis & Resolution: 1. **P0 - GRC & Survival:** The European data regulation overhaul is immediately prioritized as the top program. If you are suspended in the EU, the IPO fails entirely. Compliance risk supersedes all ROI metrics. You allocate necessary engineering capital here instantly. 2. **P1 - Financial (EBITDA):** To hit the CFO's 10% EBITDA target, you cannot rely on the delayed high-ROI project. Instead, you look to OPEX reductions. You order an immediate freeze on low-tier cloud spending, cut underperforming marketing integrations, and potentially delay the rollout of costly adjacent portfolio features. 3. **P2 - Execution & Tech Debt:** For the delayed project, you do not force a death march. You apply CPM and PERT analysis to find a slimmed-down MVP (Minimum Viable Product) that skips the tech debt hurdles temporarily, delivering *just enough* value to show momentum for the IPO, with a firm Jira Epic created to pay down the debt post-IPO. 4. **Stakeholder Comms:** You schedule an emergency Portfolio Review with the CEO and CFO, presenting the regulatory shift as an unmovable constraint, proposing the OPEX cuts to save the EBITDA target, and setting realistic expectations for the delayed project.